

Building a Basic HTTP Server

Using Node.js `http.createServer()` — Theory & Code Reference

1. What is the `http` module?

Node.js ships with a set of built-in modules that require no installation. The **http** module is one of them, and it provides the core building blocks for creating web servers and handling HTTP communication directly — without needing any external framework such as Express.js.

2. What is a server, conceptually?

A server is a program that continuously **listens** for incoming requests (from browsers, mobile apps, or tools like curl/Postman) and **responds** to them. It runs indefinitely, waiting for connections on a specific network port, and reacts each time a client makes contact.

3. The Program

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, {
    'Content-Type': 'text/plain',
    'X-Powered-By': 'Node.js HTTP Module'
  });

  res.end('Hello World');

  console.log('Status Code:', res.statusCode);
  console.log('Request Headers:', req.headers);
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}/`);
});
```

4. Key Concepts Explained

4.1 `http.createServer(callback)`

This method creates a server instance. The callback function passed into it acts as a **request handler** — it executes automatically every single time a client sends a request to the server. This callback always receives two objects:

- **req (Request object)** — contains all information about the incoming request: the URL requested, the HTTP method (GET, POST, etc.), and the request headers.
- **res (Response object)** — the object used to construct and send a reply back to the client.

4.2 HTTP Status Codes

A status code is a 3-digit number that communicates the outcome of a request to the client.

Range	Meaning	Example
2xx	Success	200 OK
3xx	Redirection	301 Moved Permanently
4xx	Client error	404 Not Found
5xx	Server error	500 Internal Server Error

4.3 HTTP Headers

Headers are metadata sent alongside the response body. They do not contain the actual content — they describe it. For example, **Content-Type: text/plain** tells the receiving client how to interpret the data that follows (as plain text, HTML, JSON, etc.). Custom headers, such as X-Powered-By in the example, can also be added purely for informational purposes.

4.4 `res.writeHead(statusCode, headersObject)`

This method sets the status code and headers together, in a single call. It must be called **before** sending any response body — once the body starts transmitting, headers can no longer be modified. This mirrors how HTTP actually works: headers always precede the body in a real HTTP response.

4.5 `res.end(data)`

This sends the response body to the client and signals that the response is complete. If this is never called, the client's request will hang indefinitely, since the server never signals that it has finished responding.

4.6 `req.headers`

An object containing everything the client sent with its request — browser type, accepted content types, cookies, authorization tokens, and more. Useful for inspecting or authenticating incoming requests.

4.7 `server.listen(port, callback)`

Defining server behavior with `createServer()` does not start the server — it only prepares it. Calling `.listen()` is what actually binds the server to a network port and begins accepting real connections. The callback runs once the server is successfully up.

5. The Request-Response Cycle

This exercise demonstrates the fundamental request-response cycle underlying all web servers, regardless of language or framework:

1. Client sends a request
2. Server receives it and processes it (via the callback)
3. Server sets the status code and headers
4. Server sends back a body
5. Client receives and interprets the response

6. Why This Matters

This is the raw, unopinionated way HTTP servers work in Node.js — no shortcuts, no magic. Every route, status code, and header must be set manually. This is precisely why **Express.js** exists: it wraps this repetitive boilerplate (writeHead, manual routing, JSON handling) into simple methods such as `res.status(200).json({...})` and `app.get('/about', handler)`. Teaching the raw http module first is valuable because it shows students exactly what a framework is doing internally, rather than treating it as an unexplainable black box.

Prepared as a lecture reference for Node.js / Web Servers module.